

HelixQA

HelixQA

Аркестрацыя анты-блефу ў кантролі якасці — аўтаномныя крос-платформавыя сесіі, дзе кожны ПРАЙШОЎшы тэст суправаджаецца захопленымі доказамі таго, што рэальны карыстальнік можа карыстацца функцыяй.

Зводка

HelixQA — гэта фрэймворк аркестрацыі анты-блефу ў кантролі якасці для крос-платформавага тэсціравання (Android, Android TV, Web, Desktop), які аб'ядноўвае банкі тэстаў YAML, выяўленне крашаў у рэжыме рэальнага часу, захоп доказаў паэтапна і аўтаномныя сесіі кантролю якасці LLM з выкарыстаннем камп'ютарнага зроку для пацвярджэння працаздольнасці функцый ад пачатку да канца. Гэта абавязковы тып тэстаў для Constitution (§11.4.169).

Кароткае апісанне

Аркестратар анты-блефу ў кантролі якасці (Go), які запускае напісаныя банкі тэстаў і цалкам аўтаномныя сесіі кантролю якасці на аснове LLM і камп'ютарнага зроку на розных платформах — выяўляе крашы, правярае кожны этап на аснове захопленых доказаў (здымкі экрана, logcat, відэа, stack traces) і аўтаматычна генеруе дэталёвыя заяўкі для канвеераў выпраўленняў AI.

Падрабязнае апісанне

HelixQA — гэта фрэймворк Go, адзіным і няўхільным прынцыпам дызайну якога з'яўляецца Аператыўнае правіла Constitution (§11.4): планка для выпуску прадукту — не “тэсты прайшлі”, а “карыстальнікі могуць карыстацца функцыяй”, таму кожны ПРАЙШОЎшы тэст павінен суправаджацца

рэальнымі доказаўмі, захопленымі падчас выканання — без доказаў няма зялёнага святла, без выключэнняў. Ён працуе ў двух дадатковых рэжымах, якія разам пакрываюць як сцэнарнае, так і невядомае. Першы — **напісаныя банкі тэстаў**: наборы YAML з тэс-XXX выпадкаў, якія ўключаюць прывязку да платформы, прыярытэты, этапы ў пэўным парадку (назва/дзеянне/чаканы вынік), тэгі і спасылкі на дакументацыю. Яны выконваюцца з праверкай кожнага этапу, выяўленнем крашаў/ANR у рэжыме рэальнага часу (ADB для Android, маніторынг працэсаў для web/desktop), цэнтралізаваным зборам доказаў і аўтаматычна сфарміраванымі заяўкамі ў фармаце Markdown для наступнай апрацоўкі ў канвееры выпраўленняў AI. Другі — **цалкам аўтаномная сесія кантролю якасці**, якая перадае дадзаткі агентам на базе LLM і камп'ютарнага зроку і дазваляе ім кіраваць ім без удзелу чалавека праз чатыры дысцыплінаваныя фазы: падрыхтоўка (выбар LLM, стварэнне карты функцый на аснове праектнай дакументацыі, запуск агентаў CLI, ініцыялізацыя рухавіка камп'ютарнага зроку), праверка на аснове дакументацыі, якая ахоплівае ўсе дакументаваныя функцыі, даследчая фаза, якая наўмысна правярае крайнія выпадкі і недакументаваныя паводзіны, а затым фарміраванне справаздачы і ачыстка ў фармаце Markdown/HTML/JSON з прывязкай кожнага знаходжання да доказаў з пазначаным часам у відэа.

Крытычна важна, што ён не ацэньвае сам сябе: інтэгруе чатыры знешнія падмадулі Go (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) і выкарыстоўвае агульную інфраструктуру challenges і containers, таму кампанент, які кіруе дадаткам, не з'яўляецца тым жа кампанентам, які ацэньвае, ці працуе ён. Уласны набор тэстаў фрэймворка падпарадкоўваецца тым самым патрабаванням, якія ён выстаўляе іншым праз каманду make anti-bluff (статычнае сканаванне + маніфест паводзінных якараў + мутацыйны рачат) і Challenge аркестратара з убудаванай мутацыяй §1.1. Матрыца пакрыцця тыпаў тэстаў на 15 радкоў звязвае кожную дэклараваную магчымасць з канкрэтным выканальным актывам і пэўнай формай захопленых доказаў — такім чынам, заявы фрэймворка аб сабе звязаны доказаўмі гэтак жа жорстка, як і вердыкты, якія ён выдае прадуктам, што тэсціруюцца.

Змест

Чаму мы яго стварылі

Звычайны кантроль якасці дае зялёнае святло паводле прынцыпу «асерцыя прайшла», і менавіта так у сістэму пракрадаецца тое, што класа Constitution называе *блефаваннем* — функцыя, пра якую паведамляюць як пра працуючую, хаця для сапраўднага карыстальніка яна

зламаныя. HelixQA быў створаны менавіта для таго, каб гэта стала немагчымым для кантролю якасці: ён адмаўляецца прысвойваць статус ПРАЙШОЎ без фізічных доказаў (здымак экрана, логкат, відэа, стэк-трэйс, справаздача), атрыманых падчас рэальнага выканання, і разглядае з’яўлены радок у справаздачы без такіх доказаў як крытычны дэфект, роўны адсутнасці функцыі. Ён таксама вырашае праблему працоўнай сілы — поўнамаштабны ручны кантроль якасці на мностве платформаў не маштабуецца — пераводзячы сесіі на поўную аўтаномнасць.

Чаму гэта пераварот у гульні

Ён аб’ядноўвае дзве рэчы, якія амаль ніколі не суіснуюць у адным інструменце: строгі кантроль якасці на аснове доказаў і аўтаномнае, самакіраўнае даследаванне. Агент LLM з функцыяй камп’ютарнага зроку адкрывае *сапраўдную* праграму, правярае кожную дакументаваную функцыю, шукае недакументаваныя памылкі, для якіх ніхто не напісаў тэст, і стварае доказы судовай якасці падчас гэтага — такім чынам, фраза «мы гэта пратэставалі» замяняецца на «вось відэа, вось логкат, вось заяўка». А паколькі гэта падсістэма кантролю якасці, названая ў Constitution, яе ўкараненне не проста павышае сумленнасць кантролю якасці для адной каманды — яно падвышае планку для кожнага прадукту ў сямействе адным крокам.

Што новага

- **Кантракт на доказы супраць блефу** — кожная праверка ПРАЙШОЎ павінна мець прывязаныя да яе доказы, атрыманыя падчас выканання; з’яўлены радок у CI лічыцца неабходнай, але ніколі не дастатковай умовай, а з’яўленая справаздача без доказаў разглядаецца як крытычны дэфект.
- **Аўтаномнае даследаванне па дакументацыі і з дапамогай цікаўнасці** — ён правярае кожную дакументаваную функцыю і потым сыходзіць са сцэнарыя, даследуючы крайнія выпадкі, з якімі сутыкаюцца сапраўдныя карыстальнікі (пустыя ўводы, хуткія ўзаемадзеянні, недакументаваныя шляхі), якія не былі прадугледжаны ні ў адным ручным наборы тэстаў.
- **Аракул камп’ютарнага зроку** — механічнае зрэненне GoCV разам з LLM Vision API літаральна *бачыць* інтэрфейс на экране, выяўляючы візуальна пашкоджаныя станы, якія прапускаюць асерцыі на ўзроўні токенаў і ўласцівасцей.
- **Банкі тэстаў на аснове структуры, а не тэксту** — радкі ў банку апісваюць структуру і генеруюць пытанні для LLM падчас выканання (CONST-046), таму адзін банк працуе для

ўсіх лакалізацый замест таго, каб разбурацца пры перакладзе тэксту інтэрфейсу.

- **Заяўкі, створаныя для канвеераў выпраўленняў AI** — аўтаматычна згенераваныя заяўкі ў фармаце Markdown паступаюць з поўным наборам доказаў, гатовыя да перадачы непасрэдна агенту па выпраўленнях, а не чалавеку-дыспетчару.

Як яго выкарыстоўваюць ва ўсіх прадуктах (магчымасці, якія ён дае)

Як абавязковая якасная апора (Constitution §11.4.169 называе падсістэму `helix_qa` адным з абавязковых тыпаў тэстаў), HelixQA надае кожнаму прадукту ў сямействе аднолькавы набор магчымасцей:

- **Аўтаномныя сесіі кантролю якасці:** адна каманда `helixqa autonomous --project ... --platforms android,desktop,web` запускаяе агента LLM з камп'ютарным зрокам, які кіруе рэальнымі праграмамі без удзелу чалавека для дасягнення мэты пакрыцця, генеруючы справаздачы, заяўкі і відэа без удзелу чалавека.
- **Банкі тэстаў / наборы:** банкі YAML (паверхня 219 з мінімальным паказчыкам ≥ 30), накіраваныя на пэўныя платформы, з прыярытэтамі і адсочваннем радок за радком да дакументацыі, якую яны правяраюць.
- **Атрыманыя доказы:** здымкі экрана, логкат, відэа, стэк-трэйсы і поўная храналагічная шкала — цэнтралізаваныя і звязаныя з кожнай справаздачай, каб любы вынік можна было паўтарыць і праверыць постфактум.
- **Незалежныя вердыкты (§11.4.141 прынцып незалежнасці):** яго `issuetelector` на аснове LLM і аракул камп'ютарнага зроку ацэньваюць паводзіны запушчанай праграмы незалежна ад агента, які ёю кіраваў, што структурна выключае класічную памылку, калі сістэма прызнае правільнай сваю ўласную працу.
- **Бар'ер і храпавы механізм мутацый:** `make qa-all / make anti-bluff i challenges/scripts/helixqa_orchestrator_challenge.sh` (8 фаз, убудаваная мутацыя §1.1) бесперапынна правяраюць сумленнасць самога HelixQA — і наўмысна не прадугледжана «лазёўкі» `--skip-helixqa`, каб адключыць дысцыпліну пад ціскам дэдлайнаў.

Змест

Найбуйнейшыя тэхнічныя праблемы і спосабы іх вырашэння

- **Папярэджанне ілжывых станоўчых вынікаў у кантролі якасці** — інструмент, які выяўляе падман, не павінен сам стаць падманшчыкам → кожны этап правяраецца на падставе сабраных доказаў, вынік PASS без доказаў фіксуецца як дэфект, а не як станоўчы вынік, і маніфест паводзінных якараў звязвае кожную дэклараваную магчымасць з выканальным тэстам (CONST-035), каб ні адна магчымасць не магла быць заяўлена без яе практычнага праверкі.
- **Кіраванне рознымі платформамі з аднаго цэнтра** — Android, Android TV, Web і Desktop не маюць агульнай мадэлі ўводу → адзіны пакет navigator абстрагуе платформаспецыфічныя ActionExecutors (ADB, Playwright, X11) і дэтэктары збояў для кожнай платформы (android/web/desktop), каб логіка аркестрацыі пісалася адзін раз, а адрозненні паміж платформамі заставаліся на ўзроўні дэталей.
- **Зрабіць аўтаномных агентаў карыснымі, а не хаатычнымі** — некантраляваны LLM у дадатку можа блукаць бясконца → LLMsVerifier ацэньвае і выбірае правільныя мадэлі, LLMOrchestrator кіруе бязгалоўнымі агентамі CLI (opencode, claude-code, gemini, junie, qwen-code), DocProcessor стварае карту функцый, якая надае даследаванню мэту, а VisionEngine трымае кожнае рашэнне ў межах рэальных пікселяў на экране, а не ўяўленняў мадэлі.
- **Бяспечныя для лакалізацыі банкі тэстаў** — набор, які жорстка прапісвае англійскі тэкст інтэрфейсу, ламаецца ў пятнаццаці мовах → банкі апісваюць толькі структуру, а тэкст для карыстальніка загружаецца праз LLM/рэсурсы падчас выканання (CONST-046), каб адзін і той жа банк правяраў аднолькавую паводзіну незалежна ад лакалі.
- **Доказ таго, што бар'еры не з'яўляюцца падманам** — антыпадманны бар'ер, які сам не можа праваліцца, — гэта найвялікшы падман → парныя мутацыі §1.1 выдаляюць захоп доказаў ці антыпадманнае сцвярджэнне тыпу і патрабуюць правалу бар'ера, а мутацыйны храпавы механізм не дазваляе гэтай гарантыі ціха знікнуць з часам.

Тэхнічны стэк

- **Аркестратар Go 1.24+** — чаму: кантроль якасці павінен працаваць усюды, дзе працуюць прадукты, таму адзіны статычна звязаны, хуткі і пераносны бінарны файл лепш за альтэрнатыву з цяжкім рантаймам; як: адзін CLI cmd/helixqa

з кампазітнымі падкамандамі `run / list / report / autonomous / version`.

- **Банкі тэстаў YAML (pkg/testbank)** — чаму: наборы павінны быць дэкларатыўнымі і зразумелымі, рэдагаванымі чалавекам без умяшання ў Go; як: `version/name/test_cases[]` з `id`, `category`, `priority`, `platforms`, упарадкаванымі `steps[]` і `documentation_refs[]` для адсочвання сувязі з дакументацыяй функцый.
- **Дэтэктары збояў/ANR (pkg/detector)** — чаму: найбольш крытычныя памылкі — тыя, што ўзнікаюць у працэсе ўзаемадзеяння, а не ў постфактумных сцвярджэннях; як: ADB (`pidof/logcat/screenshot`) для Android і `pgrep` для `web/desktop`, якія сочаць за працэсам падчас кіравання тэстам.
- **Збор доказаў (pkg/evidence, pkg/session)** — чаму: антыпадманны кантракт рэальны толькі тады, калі кожны вынік PASS падмацаваны фізічнымі доказамі; як: здымкі экрана, `logcat`, відэа і стэк-трэйсы захоўваюцца ў храналагічнай паслядоўнасці `SessionRecorder`, да якой вядуць усе справаздачы.
- **Аўтаномная сесія (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — чаму: поўнамаштабны ручны кантроль якасці на чатырох платформах не маштабуецца, таму даследаванне павінна кіравацца самастойна; як: 4-фазавы `SessionCoordinator` разам з `ActionExecutors` (ADB/Playwright/X11) і дэтэктарамі памылак LLM, якія ахопліваюць візуальныя, UX, даступнасныя і функцыянальныя дэфекты.
- **Знешнія субмадулі** — чаму: паўторнае выкарыстанне і дэкаплінг (CONST-051), а таксама — крытычна — аддзяленне навігатора ад ацэншчыка; як: `LLMsVerifier` (ацэнка мадэляў), `LLMOrchestrator` (бязгалоўныя агенты CLI), `VisionEngine` (GoCV + LLM Vision), `DocProcessor` (карта функцый/пакрыцця), кожны з якіх з'яўляецца незалежным кампанентам.
- **Антыпадманныя бар'еры + мутацыйны храпавы механізм** — чаму: каб HelixQA адпавядаў тым самым патрабаванням §1.1, якія ён накладвае на ўсё астатняе; як: скан `make anti-bluff` разам з маніфестам паводзінных якараў і мутацыйным храпавым механізмам, з `helixqa_orchestrator_challenge.sh` як 8-фазавым энда-ту-энд валідатарам.
- **Матрыца пакрыцця на 15 радкоў (docs/test-coverage.md)** — чаму: CONST-050(B) патрабуе закрытага, поўнаасцю ўлічанага набору тыпаў тэстаў без прабелаў; як: кожны радок звязаны з канкрэтным выканальным актывам і пэўнай формай сабраных доказаў, таму пакрыццё з'яўляецца правяраным фактам, а не проста сцвярджэннем.

Змест

Заўвагі аб статусе і сумленнасці

- **Статус: бэта.** Актыўна распрацоўваецца (банер статусу ў README, версія 219). Адпавядае ўласным анты-блеф стандартам.
- **Ліцэнзія: Apache-2.0.** Усталяваць: `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Прыярытэтны ўзровень: Helix-асноўны — абавязковая якасная/анты-блеф апора, якая вызначае, як сямейства Helix правярае сапраўднасць працы функцый.